

Luron AI B2B SaaS Platform

Technical Implementation Report

Executive Summary

Luron AI is a comprehensive B2B SaaS platform that enables businesses to deploy AI-powered customer service agents across multiple communication channels. The platform provides automated customer interactions through WhatsApp, SMS, and voice calls, with real-time monitoring and management capabilities through a modern web dashboard.

The system leverages a microservices-oriented architecture combining Django for backend services, Next.js for the frontend application, and integrates with third-party services including OpenAI for AI processing, Twilio for communications, and ElevenLabs for text-to-speech and speech-to-text capabilities.

This technical report represents the current state of the Luron AI platform implementation and serves as a comprehensive overview for technical stakeholders and potential investors.

Table of Contents

Luron AI B2B SaaS Platform	1
Technical Implementation Report	1
Executive Summary	1
1. Technical Stack	3
Backend Technologies	3
Frontend Technologies	3
Third-Party Integrations.....	3
2. System Architecture	4
High-Level Architecture	4
Message Flow Architecture	4
3. Core Components Implementation.....	5
3.1 Database Models	5
3.2 Service Layer	5
3.3 Real-time Communication	5
4. Completed Features	6
Phase 1: Backend Foundation 	6
Phase 2: Service Integration 	6
Phase 3: Real-time Infrastructure 	6
Phase 4: Frontend Foundation 	6
5. Features Planned for Future Phases.....	7
Phase 5: Agent Management UI	7
Phase 6: Conversation Management UI	7
6. Deployment Architecture	10
PROD Deployment Configuration	10
7. Security Implementation	10
Authentication & Authorization.....	10
Future Data Security Prospects	10
Future Compliance Considerations	10
8. Performance Optimizations.....	11
Backend Optimizations.....	11
Frontend Optimizations	11
9. Monitoring & Observability	11
Application Monitoring.....	11
Business Metrics	11
10. Development Workflow	11
Version Control	11
CI/CD Pipeline.....	11
Documentation	12
Conclusion.....	12
Technical Metrics	12
Desired System Capabilities	12
Estimated Cost Analysis (Per 1000 Conversations).....	12

1. Technical Stack

Backend:

- **Framework:** Django 4.2+ with Django Ninja for RESTful APIs
- **Database:** PostgreSQL for primary data storage (PROD)
- **Cache/Message Broker:** Redis for caching and WebSocket channel layer
- **Real-time Communication:** Django Channels for WebSocket support
- **Authentication:** Django Authentication Layer with JWT token implementation
- **Async Processing:** Python asyncio for non-blocking operations

Frontend:

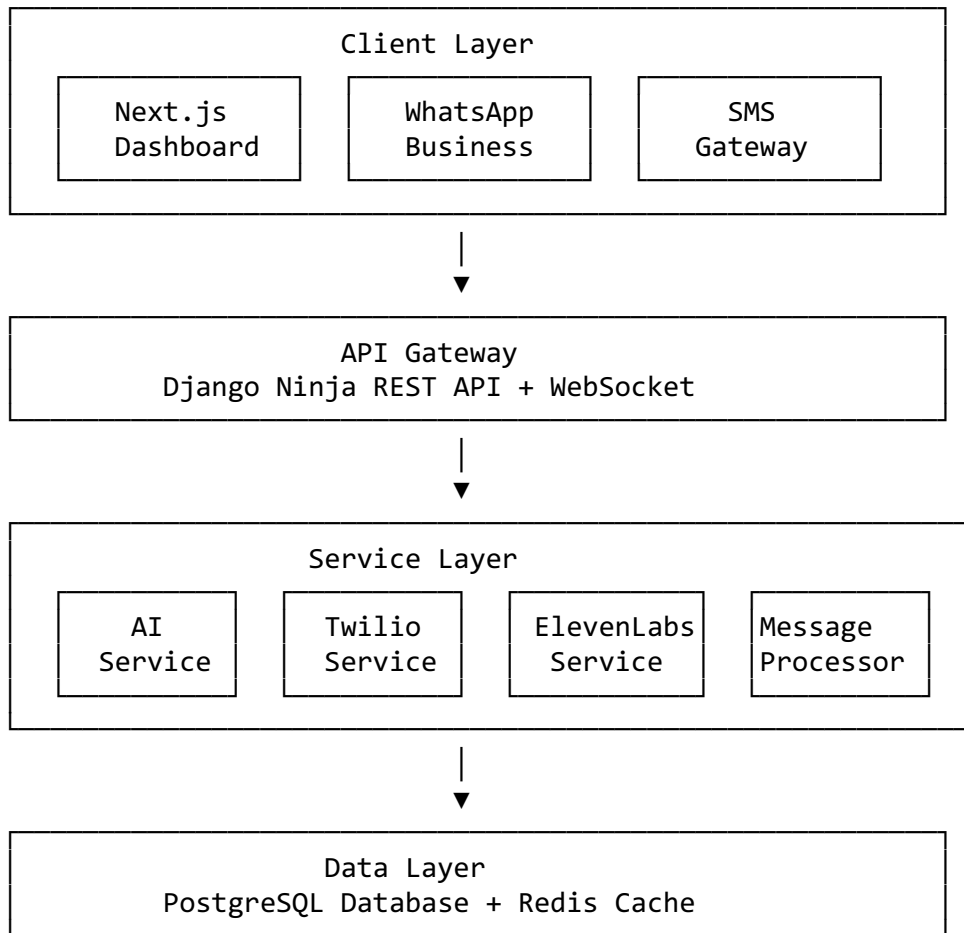
- **Framework:** Next.js 14 with App Router architecture
- **Language:** TypeScript for type safety
- **Styling:** Tailwind CSS with shadcn/ui component library
- **State Management:** Zustand for global state, React Query for server state
- **Real-time:** Socket.io-client for WebSocket connections
- **Form Handling:** React Hook Form with Zod validation

Third-Party Integrations

- **AI/LLM:** OpenAI APIs with various models for natural language processing
 - **Communications:** Twilio for WhatsApp Business API, SMS, and Voice
 - **Text-to-Speech/ Speech-to-Text:** ElevenLabs API for voice synthesis
 - **DevOps:** Railway for cloud hosting and PostgreSQL database
 - **CI/CD:** Git & Github Actions
-

2. System Architecture

High-Level Architecture



Message Flow Architecture

Customer → Channel (WhatsApp/SMS/Voice) →
Twilio Webhook → Django Backend →
Message Processor → AI Service →
Response Generation → Channel Adapter →
Twilio API → Customer

3. Core Components Implementation

3.1 Database Models

The system implements a multi-tenant architecture with the following core models:

- **User & Organization:** Multi-tenant user management with role-based access control
- **Agent:** AI agent configuration including personality, system prompts, and channel settings
- **Conversation:** Tracks all customer interactions across channels
- **Message:** Stores individual messages with delivery status tracking
- **KnowledgeDocument:** Manages uploaded documents for agent knowledge base

3.2 Service Layer

AI Service

- Integrates with OpenAI GPT-4 API (can be modified as desired)
- Manages conversation context and history
- Implements system prompt generation from agent configuration
- Handles knowledge retrieval from uploaded documents
- Provides intent classification and entity extraction
- Handles function calling based on intent

Twilio Service

- **WhatsApp Integration:** Handles WhatsApp Business API messaging
- **SMS Management:** Processes SMS with automatic message chunking for long texts
- **Voice Processing:** Implements TwiML response generation for voice calls
- **Webhook Validation:** Ensures secure webhook processing with signature verification

ElevenLabs Service

- Text-to-speech conversion for voice calls
- Voice persona selection
- Audio file caching and URL generation
- Stream processing for real-time voice synthesis

3.3 Real-time Communication

WebSocket Implementation

- Django Channels with Redis backend
- Organization-based room management
- Real-time message broadcasting
- Connection lifecycle management

- Automatic reconnection handling

Event Types

- New message notifications
 - Conversation status updates
 - Agent availability changes
 - Typing indicators
 - Read receipts
-

4. Completed Features

Phase 1: Backend Foundation

- **Multi-tenant Architecture:** Complete organization and user management system
- **Database Schema:** All core models implemented and migrated
- **Authentication System:** JWT-based authentication with refresh tokens
- **API Framework:** Django Ninja REST APIs with proper serialization
- **Admin Interface:** Django admin customized for internal management

Phase 2: Service Integration

- **OpenAI Integration:** Full GPT-4 integration with conversation context
- **Twilio Configuration:** WhatsApp, and Voice channels operational, SMS pending (code ready for testing, No SMS number could be verified/configured)
- **ElevenLabs Setup:** Text-to-speech with voice selection
- **Message Processing Pipeline:** Unified message handling across channels
- **Webhook Endpoints:** Secure webhook processing for all channels

Phase 3: Real-time Infrastructure

- **WebSocket Server:** Django Channels configured with Redis
- **Real-time Updates:** Live conversation updates to dashboard
- **Connection Management:** Automatic reconnection and error handling
- **Event Broadcasting:** Organization-scoped message broadcasting

Phase 4: Frontend Foundation

- **Next.js Setup:** TypeScript-enabled Next.js 14 application
 - **Authentication Flow:** Login, registration, and token management
 - **Dashboard Layout:** Responsive dashboard with navigation
 - **API Client:** Axios-based API client with interceptors
 - **Component Library:** Tailwind CSS with shadcn/ui components
-

5. Features Planned for Future Phases

Phase 5: Agent Management UI

- **Agent Dashboard:** Visual interface for agent configuration (add, edit, delete, activate, deactivate, configure) ✓
 - **Agent Voice Configuration:** Voice preview and selection
 - **Testing Interface:** Built-in testing for messages and calls
 - **Template System:** Pre-configured agent templates
-

Phase 6: Conversation Management UI

Backend

- Conversation search API
- Advanced analytics
- Export functionality
- Batch operations

Frontend

- **Conversation Interface**
 - Split-view layout
 - Real-time messages (using Phase 3 WebSocket)
 - Message input
 - Media support
-

Phase 7: Analytics & Monitoring

Backend

- Metrics aggregation (APIs ready)
- Report generation
- Time-series data
- Export APIs

Frontend

- **Analytics Dashboard**
 - Overview metrics
 - Charts and graphs
 - Channel distribution
 - Agent performance

- **Usage & Billing**
 - Current usage
 - Cost breakdown
 - Invoice history
-

Phase 8: Advanced AI Features

Backend (Stack most likely: Pycone Vector Database, LangChain)

- Vector embeddings
- Semantic search
- Intent classification
- Conversation summarization
- Multi-language support

Frontend

- **AI Configuration**
 - Advanced prompt editor
 - Model selection
 - Parameter tuning
 - Training interface
-

Phase 9: Team & Collaboration

Backend

- Team invitation system
- Role permissions
- Activity logging
- Audit trail

Frontend

- **Team Management**
 - Invite members
 - Role assignment
 - Activity feed
 - Permissions UI
-

Phase 10: Production Optimization

Backend

- Query optimization
- Caching layer
- Background jobs (Celery)
- Rate limiting
- Error handling

Frontend

- Code splitting
 - Performance optimization
 - PWA features
 - Dark mode
-

Phase 11: Integrations

Backend

- CRM connectors
- Slack integration
- Email channel
- Zapier webhooks
- Public API

Frontend

- Integration marketplace
 - OAuth flows
 - Webhook management
 - API documentation
-

6. Deployment Architecture

PROD Deployment Configuration

The application is designed for deployment on Railway with the following configuration:

- **Backend Service:** Django application with Daphne ASGI server
 - **Frontend Service:** Next.js application with Node.js runtime
 - **Database:** PostgreSQL managed instance
 - **Redis:** Managed Redis instance for caching and channels
 - **Static Assets:** Served via CDN
 - **Environment Management:** Separate development, staging, and production environments
-

7. Security Implementation

Authentication & Authorization

- JWT token-based authentication (implemented)
- Role-based access control (RBAC)
- Organization-level data isolation
- Refresh token rotation (implemented)

Future Data Security Prospects

- HTTPS enforcement for all communications
- Webhook signature verification
- Input validation and sanitization
- SQL injection prevention through ORM
- XSS protection in frontend

Future Compliance Considerations

- GDPR-compliant data handling
 - User consent management
 - Data retention policies
 - Audit trail for sensitive operations
-

8. Performance Optimizations

Backend Optimizations

- Database query optimization with `select_related` and `prefetch_related`
- Async/await for I/O operations
- Connection pooling for external services
- Response caching for frequently accessed data

Frontend Optimizations

- Server-side rendering for initial page load
 - Code splitting and lazy loading
 - Image optimization
 - Bundle size optimization
 - Progressive Web App capabilities
-

9. Monitoring & Observability

Application Monitoring

- Error tracking and logging
- Performance metrics collection
- API endpoint monitoring
- WebSocket connection tracking

Business Metrics

- Conversation volume tracking
 - Response time analytics
 - AI model performance metrics
 - Cost per conversation calculations
-

10. Development Workflow

Version Control

- Git-based version control
- Feature branch workflow
- Pull request reviews

CI/CD Pipeline

- Automated testing suite

- Linting and code quality checks
- Staging deployment for testing
- Production deployment with rollback capability

Documentation

- API documentation with OpenAPI/Swagger
 - Code documentation with docstrings
 - Architecture decision records
 - User guides and tutorials
-

Conclusion

The Luron AI platform represents a comprehensive solution for B2B customers seeking to implement AI-powered customer service across multiple channels. The architecture is designed for scalability, maintainability, and extensibility, with clear separation of concerns and modern development practices.

The phased implementation approach allows for iterative development and testing, ensuring each component is robust before moving to the next phase. The current implementation provides a solid foundation with core messaging capabilities, real-time updates, and AI integration fully operational.

Future phases will focus on enhancing the user experience, adding advanced analytics, and implementing enterprise-grade features to support larger organizations and more complex use cases.

Technical Metrics

Desired System Capabilities

- **Concurrent Conversations:** 1000+ simultaneous conversations
- **Message Processing Time:** <2 seconds average response time
- **Uptime Target:** 99.9% availability
- **API Response Time:** <200ms for standard endpoints
- **WebSocket Latency:** <100ms for real-time updates

Estimated Cost Analysis (Per 1000 Conversations)

- **Messaging Costs:** ~\$50-80 (WhatsApp/SMS)
- **AI Processing:** ~\$200 (GPT-4)
- **Voice Synthesis:** ~\$100 (ElevenLabs)

- **Infrastructure:** ~\$50-100 (Railway)
 - **Total Estimated Cost:** ~\$500-600
-